

---

# 基于采样的推理： 您的基础模型比想象中更智能

---

Aayush Karan<sup>1</sup>, Yilun Du<sup>1</sup>

<sup>1</sup>Harvard University

[Website](#) [Code](#)

## Abstract

前沿推理模型通过强化学习（RL）对大型语言模型（LLMs）进行后训练，已在多学科领域展现出卓越的能力。然而，尽管这一范式取得了广泛成功，现有研究大多聚焦于解析RL阶段涌现而基础模型不具备的全新行为。本研究另辟蹊径，转而探究是否仅通过纯采样方式——无需任何额外训练——即可在推理阶段从基础模型中激发出与之匹敌的推理能力。受马尔可夫链蒙特卡洛（MCMC）从锐化分布中采样技术的启发，我们提出一种基于基础模型自身似然的简易迭代采样算法。实验表明，在不同基础模型上，该算法对单次任务（包括MATH500、HumanEval和GPQA等）的推理能力提升显著，其效果近乎匹配甚至超越RL后训练结果。此外，我们的采样器避免了RL后训练典型的多样本多样性衰减问题。值得注意的是，该方法无需训练过程、精选数据集或验证器，预示着其在可验证领域之外的广泛应用潜力。

## 1 引言

强化学习（RL）已成为增强大语言模型（LLMs）推理能力的主导范式 [Guo et al., 2025, Hu et al., 2025]。通过配备通常可自动验证的奖励信号，主流RL技术已成功应用于前沿模型的后期训练，在数学、编程和科学等领域带来显著的性能提升 [Hendrycks et al., 2021, Li et al., 2022, Rein et al., 2024]。

尽管RL在LLMs中取得了广泛的实证成功，大量文献仍围绕以下问题展开：RL后期训练中出现的能力究竟是基础模型原本不具备的全新行为，还是已有能力的重新分配？这就是分布锐化问题 [He et al., 2025, Shao et al., 2025, Yue et al., 2025]——即后期训练后的分布究竟是基础模型分布的“锐化版”，还是能生成基础模型几乎不可能产生的推理轨迹。

多项研究指出了RL后期训练在学习新能力方面的困难。He et al. [2025], Song et al. [2025] 比较了基础模型与后期训练模型在pass@ $k$ （多样本）指标上的表现，发现当 $k$ 较大时，基础模型反而优于后期训练模型，后者存在生成多样性下降的问题。这种情况下，RL似乎以牺牲多样本推理能力为代价，将pass@ $k$ 性能重新分配到单样本性能上。Yue et al. [2025] 也指出，经过RL训练后的推理轨迹在基础模型下具有极高的似然值/置信度，似乎是从已有高似然能力中提取的。我们在图 4 的实验中印证了这一点。尽管如此，RL后期训练对单样本推理的优势至今仍不可否认。

Preprint. Under review.

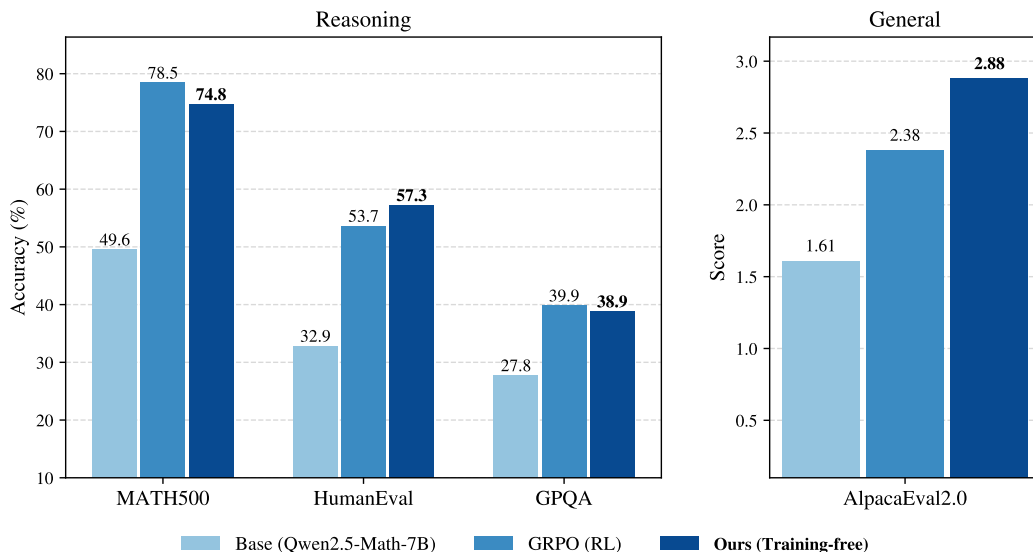


图 1: 我们的采样算法能够匹配并超越强化学习后训练方法。左图: 我们将采样算法 (ours) 与基础模型 (base) 及强化学习后训练 (GRPO) 在三个可验证推理任务 (MATH500、HumanEval、GPQA) 上进行对比。右图: 我们在一个不可验证的通用任务 (AlpacaEval2.0) 上进行比较。我们的算法在后训练领域内 (MATH500) 达到与GRPO相当的性能, 但在HumanEval和AlpacaEval等域外任务中能够超越其表现。

本文提出一个令人惊讶的发现: 直接从基础模型采样即可达到与 $RL$ 相当的单样本推理能力。

我们为基模型提出一种推理阶段利用额外计算资源的采样算法, 其单样本性能在领域内推理任务上接近 $RL$ 后期训练效果, 在领域外推理任务上甚至能超越 $RL$ 表现。更重要的是, 我们的采样器不会降低生成多样性——实际上, 我们的 $\text{pass}@k$  (多样本) 性能显著优于 $RL$ 方法。我们特别以群体相对策略优化 (GRPO) 为基准进行对比, 这是当前增强LLM推理的标准 $RL$ 算法 [Shao et al., 2024]。

关键优势在于, 我们的算法无需训练、无需数据集且无需验证器, 规避了 $RL$ 方法的固有弱点, 包括: 为避免训练不稳定性而进行的大量超参数搜索、需要构建多样且全面的后期训练数据集, 以及缺乏可靠的真实验证/奖励信号 [Prabhudesai et al., 2025]。

我们的贡献可总结如下:

- i) 我们提出将幂分布作为推理任务的有效采样目标。由于该分布可直接通过基础大语言模型 (LLM) 显式指定, 因此无需额外训练。
- ii) 我们进一步提出基于马尔可夫链蒙特卡洛 (MCMC) 的幂分布近似采样算法, 该算法根据基础模型的似然度对标记子序列进行迭代重采样。
- iii) 我们在多个模型 (Qwen2.5-Math-7B、Qwen2.5-7B、Phi-3.5-mini-instruct) 和推理任务 (MATH500、HumanEval、GPQA、AlpacaEval 2.0) 上实证验证了算法的有效性。实验结果表明, 直接从基础模型采样可获得与GRPO相当的效果。事实上, 在某些领域外任务中, 我们的算法始终优于强化学习基线。此外, 通过多次采样, 我们避免了强化学习后训练中常见的多样性坍缩问题, 在单次/少量样本推理能力和样本多样性方面实现了双赢。

研究结果共同表明: 现有基础模型在单样本推理方面的潜力远超当前采样方法所展现的水平。

## 2 相关工作

面向大语言模型的强化学习。强化学习（RL）在大语言模型（LLM）的后训练阶段发挥了关键作用。早期研究中，基于人类反馈的强化学习（RLHF）[Ouyang et al., 2022] 被开发为通过训练奖励模型使LLM与人类偏好对齐的技术。近年来，可验证奖励的强化学习（RLVR）作为一种强大的新型后训练技术兴起，多项工作 [Guo et al., 2025, Lambert et al., 2024, Hu et al., 2025, Zeng et al., 2025] 发现，由自动化验证器提供的简单生成末端奖励能显著提升模型在数学与编程等复杂推理任务上的表现。其中，组相对策略优化（GRPO）算法是这些突破的核心 [Shao et al., 2024]。基于此成功，后续研究探索了利用内部信号（如自熵 [Zhao et al., 2025]、置信度 [Prabhudesai et al., 2025]）甚至随机奖励 [Shao et al., 2025] 构建奖励机制。与这些工作类似，本文研究基础模型似然作为提升推理性能的机制，但关键区别在于我们的技术是免训练的。

大语言模型的自回归MCMC采样。先前工作探索了将经典MCMC技术与自回归采样相结合。在红队测试、提示工程和个性化生成等场景中，目标采样可视为从基础LLM分布出发但偏向外部奖励函数的倾斜采样。Zhao et al. [2024] 提出学习中间价值函数用于序贯蒙特卡罗（SMC）框架 [Chopin, 2004]，通过维护多个候选序列并根据预期未来奖励进行更新。类似地，Faria et al. [2024] 提出Metropolis-Hastings（MH）算法，通过迭代重采样而非维护多个候选序列，同样依据预期奖励更新。方法上，我们的采样算法与后者更相似，但关键区别在于目标采样分布完全由基础LLM确定，无需外部奖励。

扩散模型的退火采样。在统计物理与蒙特卡洛文献中，从 $p^\alpha$ 采样被称为退火或回火分布采样 [Neal, 1998]，这激发了扩散模型领域的新研究浪潮。传统MCMC采样中，退火用于避免采样时的模式坍塌，从而更准确地从复杂多峰分布中采样 [Łatuszyński et al., 2025]。这演变为扩散模型的推理时采样方法，旨在引导预训练模型朝向“倾斜分布” [Du et al., 2023, Kim et al., 2025, Karan et al., 2025, Wang et al., 2025, Kong et al., 2025, Zhang et al., 2025]。当传统RL技术出现模式坍塌时，物理科学应用 [Sambridge, 2014] 需要多峰采样。为此，Du et al. [2023], Wang et al. [2025], Kim et al. [2025] 等构建退火分布序列以平滑从基础扩散分布到倾斜分布的过渡。另一些工作 [Skreta et al., 2025, Xu et al., 2025] 专门针对 $\alpha > 1$ 的 $p^\alpha$ 采样，作为从基础扩散模型生成更高质量样本的手段，这在蛋白质设计生成中尤为流行 [Geffner et al., 2025]。

## 3 预备知识

设 $\mathcal{X}$ 为一个有限的词汇标记集合， $\mathcal{X}^T$ 表示有限标记序列 $x_{0:T} = (x_0, x_1, \dots, x_T)$ 的集合，其中对所有 $i$ 有 $x_i \in \mathcal{X}$ ，且 $T \in \mathbb{Z}_{\geq 0}$ 为非负整数。为方便起见，对于给定的 $t$ ，令 $x_{<t} = (x_0, \dots, x_{t-1})$ ， $x_{>t} = (x_{t+1}, \dots, x_T)$ ，类似定义 $x_{\leq t}$ 和 $x_{\geq t}$ 。一般情况下， $\mathbf{x}$ 指代标记序列 $x_{0:T}$ ，其中 $T$ 为隐式给定值。

通过自回归地学习所有 $t$ 时刻的条件标记分布 $p(x_t|x_{<t})$ ，大语言模型定义了标记序列 $\mathcal{X}^T$ 上的分布 $p$ ，并利用恒等式给出联合分布：

$$p(x_{0:T}) = \prod_{t=0}^T p(x_t|x_{<t}). \quad (1)$$

要从 $p$ 中采样序列，我们只需利用条件分布逐标记地从大语言模型采样，根据公式(1)即可直接从联合分布中采样。

## 4 幂分布的MCMC采样方法

本节将介绍针对基础模型的采样算法。我们的核心思想源自第1节提出的分布锐化概念。所谓锐化参考分布，是指通过重新加权使高似然区域获得更大权重，同时降低低似然区域的权重，从而使采样结果显著偏向参考分布中更高似然的区域。若经过强化学习微调的模型确实只是基础模型的锐化版本，那么我们应该能够显式地指定一个可实现相同效果的目标采样分布。

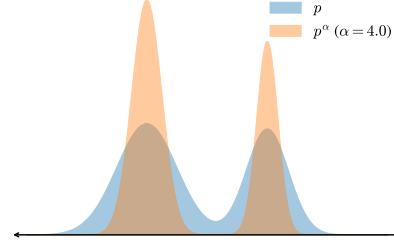


图 2: 分布锐化的一个示例。此处  $p$  为高斯混合分布，我们将其与  $p^\alpha$  ( $\alpha = 4.0$ ) 进行对比绘制。

本节内容安排如下：第4.1节提出这种目标锐化分布，并从数学角度论证其采样结果为何适用于推理任务；第4.2节介绍一类通用的马尔可夫链蒙特卡洛（MCMC）算法，旨在实际从该目标分布中采样；最后，第4.3节详述我们在大型语言模型中的具体实现方案。

### 4.1 基于幂分布的推理方法

一种自然的概率分布锐化方法是从幂分布  $p^\alpha$  中进行采样。由于

$$p(\mathbf{x}) > p(\mathbf{x}') \implies \frac{p(\mathbf{x})^\alpha}{p(\mathbf{x}')^\alpha} > \frac{p(\mathbf{x})}{p(\mathbf{x}')} \quad (\alpha \in [1, \infty]), \quad (2)$$

可知对  $p$  进行指数运算会增加高似然序列  $(\mathbf{x})$  的相对权重，同时降低低似然序列  $(\mathbf{x}')$  的相对权重（可视化效果参见图2）。

一个相关但广为人知的锐化策略是低温采样[Wang et al., 2020]，该方法在每一步对条件下一词元分布进行指数运算：

$$p_{\text{temp}}(x_t | x_0 \dots x_{t-1}) = \frac{p(x_t | x_{t-1} \dots x_0)^\alpha}{\sum_{x'_t \in \mathcal{X}} p(x'_t | x_{t-1} \dots x_0)^\alpha}, \quad (3)$$

其中温度参数为  $\tau = 1/\alpha$ 。常见的误解是认为在  $T$  个词元上使用(3)采样等价于从  $p^\alpha$  采样；然而这种认知存在微妙却关键的错误，我们将在下文阐明。

**Proposition 1.** 低温采样无法从幂分布  $p^\alpha$  中抽取样本。

证明. 我们证明两种方法在每个时间步  $t$  对应的条件下一词元分布都不相同。对于  $p^\alpha$ ， $x_t$  的条件分布为

$$p_{\text{pow}}(x_t | x_0 \dots x_{t-1}) = \frac{\sum_{x_{>t}} p(x_0, \dots, x_t, \dots, x_T)^\alpha}{\sum_{x_{\geq t}} p(x_0, \dots, x_t, \dots, x_T)^\alpha}. \quad (4)$$

根据贝叶斯规则

$$p(x_t | x_{t-1} \dots x_0) = \frac{p(x_0, \dots, x_t)}{p(x_0, \dots, x_{t-1})} = \frac{\sum_{x_{>t}} p(x_0, \dots, x_t, \dots, x_T)}{\sum_{x_{\geq t}} p(x_0, \dots, x_t, \dots, x_T)}, \quad (5)$$

我们可以将低温采样的边缘分布(3)重写为

$$p_{\text{temp}}(x_t | x_0 \dots x_{t-1}) = \frac{\left( \sum_{x_{>t}} p(x_0, \dots, x_t, \dots, x_T) \right)^\alpha}{\sum_{x'_t} \left( \sum_{x_{>t}} p(x_0, \dots, x_t, \dots, x_T) \right)^\alpha}. \quad (6)$$

为清晰起见忽略归一化项，从  $p^\alpha$  采样时词元  $x_t$  的相对权重由指数和决定

$$p_{\text{pow}}(x_t | x_{<t}) \propto \sum_{x_{>t}} p(x_0, \dots, x_t, \dots, x_T)^\alpha. \quad (7)$$

而低温采样的相对权重则由和的指数决定

$$p_{\text{temp}}(x_t | x_{<t}) \propto \left( \sum_{x_{>t}} p(x_0, \dots, x_t, \dots, x_T) \right)^\alpha. \quad (8)$$

由于两种采样策略在下一词元预测的相对权重上存在差异，因此它们在序列上的联合分布也必然不同。由此可见，低温采样得到的序列分布与 $p^\alpha$ 给出的分布并不相同。□

理解这种差异的直观方式是：低温采样没有考虑在时间步 $t$ 时指数运算如何锐化”未来路径”的似然，而是”贪婪地”平均所有这些未来似然（即和的指数(8)）。相反，从 $p^\alpha$ 采样本质地考虑了未来补全情况，因为它先对所有未来路径进行指数运算（即指数和(7)），再计算下一词元预测的权重。这将导致以下结果：

**Observation 1.** 功率分布会对具有少量但高可能性未来路径的标记进行加权，而低温采样则会对具有多个但低可能性补全结果的标记进行加权。

**Example 1.** 我们可以通过一个简单示例观察这一现象。考虑词元集合 $\mathcal{X} = \{a, b\}$ ，并限定关注双词元序列 $(x_0, x_1)$ ： $aa, ab, ba, bb$ 。设

$$p(aa) = 0.00, \quad p(ab) = 0.40, \quad p(ba) = 0.25, \quad p(bb) = 0.25,$$

则有

$$p(x_0 = a) = 0.40, \quad p(x_0 = b) = 0.50.$$

取 $\alpha = 2.0$ 时，在 $p^\alpha$ 下可得

$$p_{\text{pow}}(x_0 = a) \propto 0.00^2 + 0.40^2 = 0.160, \quad p_{\text{pow}}(x_0 = b) \propto 0.25^2 + 0.25^2 = 0.125,$$

因此 $p^\alpha$ 更倾向于采样 $a$ 而非 $b$ 。在低温采样下，

$$p_{\text{temp}}(x_0 = a) \propto (0.00 + 0.40)^2 = 0.160, \quad p_{\text{temp}}(x_0 = b) \propto (0.25 + 0.25)^2 = 0.250,$$

则更倾向于采样 $b$ 而非 $a$ 。若 $p^\alpha$ 采样 $x_0 = a$ ，则仅存在一条概率为0.40的后续路径；若 $p_{\text{temp}}$ 采样 $x_0 = b$ ，则存在 $ba, bb$ 两条后续路径，但每条路径概率均为0.25。

换言之，尽管 $a$ 在 $p$ 和 $p_{\text{temp}}$ 下都具有较低的条件概率， $p^\alpha$ 仍会提升 $a$ 的权重并采样最高概率的双词元序列。 $b$ 虽在 $p$ 和 $p_{\text{temp}}$ 下因多条后续路径而具有较高概率，但会导致低概率序列的产生。我们在附录A.1中对这一现象给出了更严格的形式化证明。

因此，从 $p^\alpha$ 采样会倾向于选择那些具有较少但更高似然”未来路径”的词元，而不是具有多个较低似然补全路径的词元。这种行为模式对于推理任务极具价值。例如，选择那些具有高平均似然但会将输出困在个别低似然未来中的”错误”词元，正是关键窗口或关键令牌[Li et al., 2025, Abdin et al., 2024]的典型示例——这种现象指少数词元对语言模型输出的正确性具有极大影响力。事实上，尖锐的关键窗口已被证明与推理失败高度相关[Li et al., 2025]。而幂分布采样中则隐含着对未来高似然词元的规划偏好。

## 4.2 Metropolis-Hastings算法

既然我们已经从理论上了解了如何通过从 $p^\alpha$ 中采样来辅助底层大语言模型（LLM）的推理能力，现在我们的目标转向提出一种能准确从中采样的算法。给定一个大语言模型 $p$ ，我们可以获取任意序列长度上 $p^\alpha$ 的值；然而，这些值是未归一化的。要从真实概率中直接采样，需要对所有序列 $(x_0, \dots, x_T) \in \mathcal{X}^T$ 进行归一化，这在计算上是不可行的。

为解决这一问题，我们采用了一种称为Metropolis-Hastings（MH）的马尔可夫链蒙特卡洛（MCMC）算法[Metropolis et al., 1953]，它恰好能满足我们的需求：从未归一化的概率分布



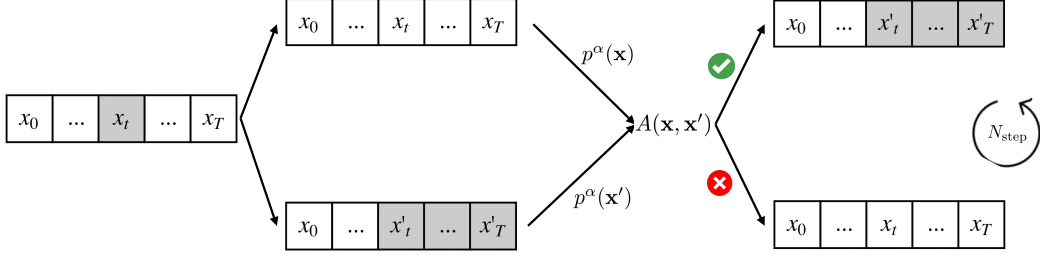


图 3: 随机重采样下的Metropolis-Hastings算法图示。随机选择索引 $t$ 并通过重采样生成新候选样本。根据相对似然值决定接受或拒绝该候选样本，该过程不断重复。

中进行近似采样。MH算法通过任意提议分布  $q(\mathbf{x}|\mathbf{x}^i)$  选择下一个候选序列  $\mathbf{x}^{i+1}$ ，从而构建一个样本序列的马尔可夫链  $(\mathbf{x}^0, \mathbf{x}^1, \dots, \mathbf{x}^n)$ 。以概率

$$A(\mathbf{x}, \mathbf{x}^i) = \min \left\{ 1, \frac{p^\alpha(\mathbf{x}) \cdot q(\mathbf{x}^i|\mathbf{x})}{p^\alpha(\mathbf{x}^i) \cdot q(\mathbf{x}|\mathbf{x}^i)} \right\}, \quad (9)$$

候选序列  $\mathbf{x}$  会被接受为  $\mathbf{x}^{i+1}$ ；否则，MH算法将设置  $\mathbf{x}^{i+1} = \mathbf{x}^i$ 。该算法特别方便，因为它仅需要  $p^\alpha$  给出的相对权重（因为  $A$  中的归一化权重会抵消），并且可以与任何通用但易于处理的采样器  $q$  配合使用，限制极少。值得注意的是，对于足够大的  $n$ ，在提议分布满足以下（非常宽松的）条件时，该过程会收敛到从目标分布  $p^\alpha$  中采样[Neal, 1993]:

**Definition 1.** 若对于目标分布  $p^\alpha$  下具有非零质量的任意集合  $X$ ，提议分布  $q$  最终都能以非零概率从  $X$  中采样，则称该提议分布是不可约的。若所诱导的样本链不会在固定间隔步数后返回相同样本，则称该提议是非周期性的。

因此，我们只需确保提议分布满足不可约性和非周期性，Metropolis-Hastings算法会处理其余部分。在实际应用中，我们还希望  $q(\mathbf{x}|\mathbf{x}^i)$  及其反向  $q(\mathbf{x}^i|\mathbf{x})$  都能轻松计算。

考虑以下随机重采样提议分布族（见图3）。设  $p_{\text{prop}}$  为提议LLM。以均匀概率  $\frac{1}{T}$  随机选择一个  $t \in [1, T]$ ，并使用  $p_{\text{prop}}$  从索引  $t$  开始重采样序列。此时，转移似然  $q(\mathbf{x}|\mathbf{x}^i)$  即为重采样的似然。注意，在每个候选选择步骤中，我们都有非零概率在任意两个序列  $\mathbf{x}, \mathbf{x}' \in \mathcal{X}$  之间转移，因为总有一定概率可以从  $\mathbf{x}$  的开头开始重采样。这确保了我们的提议分布既不可约又非周期。此外， $q(\mathbf{x}^i|\mathbf{x})$  通过对称性也易于计算，因为我们可以将  $\mathbf{x}^i$  视为  $\mathbf{x}$  的重采样版本。

借助Metropolis-Hastings算法赋予的灵活性，我们可以选择任何LLM作为提议LLM  $p_{\text{prop}}$ ，并采用任何采样策略（例如低温采样）。

### 4.3 基于自回归MCMC的幂次采样

直接为大型语言模型实现Metropolis-Hastings算法需要初始化一个长度为  $T$  的采样标记序列，随后通过多次迭代生成新的候选序列，并在每次迭代中应用公式(9)进行接受判断。然而，由于需要反复调用LLM进行完整序列推断，这一过程计算成本极高。

事实上，MCMC算法在实际应用中的主要缺陷在于可能产生指数级混合时间[Gheissari et al., 2017]——当初始化或提议分布选择不当时，可能需要指数级数量的样本才能收敛到目标分布。若样本空间具有高维特性[Bandeira et al., 2022, Schmidler and Woodard, 2013]，这一问题会进一步恶化，而标记序列空间  $\mathcal{X}^T$  恰恰呈现这种高维特性，尤其对于长序列/大  $T$  值情况。

为解决这个问题，我们提出了一种利用自回归采样序列结构的算法。我们定义了一系列中间分布，通过逐步采样最终收敛到目标分布  $p^\alpha$ 。特别地，来自某个中间分布的样本会为下一个分布启动Metropolis-Hastings过程，从而避免病态初始化。

---

**Algorithm 1:** 自回归模型的功率采样

---

**Input** : base  $p$ ; proposal  $p_{\text{prop}}$ ; power  $\alpha$ ; length  $T$ **Hyperparams:** block size  $B$ ; MCMC steps  $N_{\text{MCMC}}$ **Output** :  $(x_0, \dots, x_T) \sim p^\alpha$ **1 Notation:** Define the unnormalized intermediate target

$$\pi_k(x_{0:kB}) \propto p(x_{0:kB})^\alpha.$$

**2 for**  $k \leftarrow 0$  **to**  $\lceil \frac{T}{B} \rceil - 1$  **do****3** | Given prefix  $x_{0:kB}$ , we wish to sample from  $\pi_{k+1}$ . Construct initialization  $\mathbf{x}^0$  by extending autoregressively with  $p_{\text{prop}}$ :

$$x_t^{(0)} \sim p_{\text{prop}}(x_t \mid x_{<t}), \quad \text{for } kB + 1 \leq t \leq (k+1)B.$$

Set the current state  $\mathbf{x} \leftarrow \mathbf{x}^0$ .**4 for**  $n \leftarrow 1$  **to**  $N_{\text{MCMC}}$  **do****5** | Sample an index  $m \in \{1, \dots, (k+1)B\}$  uniformly.**6** | Construct proposal sequence  $\mathbf{x}'$  with prefix  $x_{0:m-1}$  and resampled completion:

$$x'_t \sim p_{\text{prop}}(x_t \mid x_{<t}), \quad \text{for } m \leq t \leq (k+1)B.$$

**7** | Compute acceptance ratio (9)

$$A(\mathbf{x}', \mathbf{x}) \leftarrow \min \left\{ 1, \frac{\pi_k(\mathbf{x}')}{\pi_k(\mathbf{x})} \cdot \frac{p_{\text{prop}}(\mathbf{x} \mid \mathbf{x}')}{p_{\text{prop}}(\mathbf{x}' \mid \mathbf{x})} \right\}.$$

Draw  $u \sim \text{Uniform}(0, 1)$ ;**8** | **if**  $u \leq A(\mathbf{x}', \mathbf{x})$  **then accept** and set  $\mathbf{x} \leftarrow \mathbf{x}'$ **9** | **end****10** | Set  $x_{0:(k+1)B} \leftarrow \mathbf{x}$  to fix the new prefix sequence for the next stage.**11 end****12 return**  $x_{0:T}$ 

---

固定块大小  $B$  和提议 LLM  $p_{\text{prop}}$ , 考虑以下 (未归一化的) 分布序列:

$$\emptyset \longrightarrow p(x_0, \dots, x_B)^\alpha \longrightarrow p(x_0, \dots, x_{2B})^\alpha \longrightarrow \dots \longrightarrow p(x_0, \dots, x_T)^\alpha, \quad (10)$$

其中  $p(x_0, \dots, x_{kB})$  表示长度为  $kB$  的标记序列联合分布。为方便起见, 令  $\pi_k$  表示由

$$\pi_k(x_{0:kB}) \propto p(x_{0:kB})^\alpha. \quad (11)$$

给出的分布。假设我们已获得  $\pi_k$  的样本, 为从  $\pi_{k+1}$  采样, 我们首先通过  $p_{\text{prop}}$  采样接下来的  $B$  个标记  $x_{kB+1:(k+1)B}$  来初始化 Metropolis-Hastings 过程, 随后使用前文所述的随机重采样提议分布  $q$  运行  $N_{\text{MCMC}}$  步 MCMC 采样。完整流程见算法 1。

需注意算法 1 是单次执行的: 尽管进行了多次推断调用, 但接受或拒绝新标记的决策完全基于基础模型似然, 以模拟从  $p^\alpha$  中采样单个序列的过程。这可以视为推断时扩展的新维度——我们在采样阶段投入额外计算资源以获得更高质量/更高似然的样本。

为量化扩展效果, 可以估算算法 1 生成的平均标记数。当从  $\pi_k(x_{0:kB})$  采样时, 每个候选生成步骤平均需要重采样  $\frac{kB}{2}$  个标记, 共执行  $N_{\text{MCMC}}$  次。对所有  $k$  求和后, 预期生成的标记总数

为:

$$\mathbb{E}_{\text{tokens}} = N_{\text{MCMC}} \sum_{k=1}^{\lceil T/B \rceil} \frac{kB}{2} \approx \frac{N_{\text{MCMC}} T^2}{4B}. \quad (12)$$

这里的关键权衡在于块大小 $B$ 与MCMC步数 $N_{\text{MCMC}}$ 之间的关系。较大的 $B$ 需要在中间分布间进行更大”跳跃”，因此需要更大的 $N_{\text{MCMC}}$ 来实现充分转移。在5节中，我们通过实验找到了一个 $B$ 值，使得算法1在相对较小的 $N_{\text{MCMC}}$ 值下仍能保持良好性能。

## 5 实验

### 5.1 实验设置

**评估指标。**我们采用一套标准推理基准测试集进行评估，涵盖数学、编程和STEM领域（MATH500、HumanEval、GPQA），同时使用不可验证的通用帮助性评估基准（AlpacaEval 2.0）。所有方法与基线均采用单次采样评估策略，即基于最终响应字符串进行评判。

- **MATH500:** MATH数据集 [Lightman et al., 2024] 包含涵盖几何、数论和微积分预备知识等七个类别的竞赛数学题。总计12500道题目，其中训练集7500道，测试集5000道。MATH500是OpenAI标准化的测试集中随机选取的特定子集。
- **HumanEval:** HumanEval是由164道手写编程题组成的评估集，涵盖算法、推理、数学和语言理解等领域 [Chen et al., 2021]。每道题目平均配备7.7个单元测试，解题成功需通过所有对应测试用例。
- **GPQA:** GPQA [Rein et al., 2024] 是一个包含物理、化学和生物学科的多选题数据集，需要高级推理能力才能解答。我们采用其子集GPQA Diamond进行评估，该子集包含198道代表GPQA最高质量的问题。
- **AlpacaEval 2.0:** AlpacaEval数据集包含805个提示词 [Dubois et al., 2024]，用于评估通用帮助能力，例如电影评论、推荐和邮件阅读等场景。模型响应由自动化LLM评判员（GPT-4-turbo）进行评分，判定其相对于基线模型（同为GPT-4-turbo）的偏好程度。最终得分为经响应长度归一化后的模型胜率。

**模型选择。**为验证采样算法的有效性，我们选用基础模型Qwen2.5-Math-7B、Qwen2.5-7B和Phi-3.5-mini-instruct。强化学习基线采用Shao et al. [2025]中GRPO的实现方案，这些模型均在MATH训练集上进行了后训练。对于Qwen2.5系列模型，我们沿用Shao et al. [2025]中基准测试的默认超参数。Phi-3.5模型的超参数选自Abdin et al. [2024]中能避免训练不稳定性、并通过多轮迭代实现性能提升的参数组合。

**采样算法。**在实现幂采样算法（算法1）时，设定最大生成长度 $T_{\max} = 3072$ （遇到EOS标记可提前终止），块大小 $B = 3072/16 = 192$ 。实验表明，对于推理任务，当 $\alpha = 4.0$ 且提议分布LLM  $p_{\text{prop}}$ 采用基础模型配合采样温度 $1/\alpha$ 时性能最优。在AlpacaEval 2.0评估中，提高提议分布温度（ $\tau = 0.5$ ）可进一步提升表现。

### 5.2 实验结果

**主要结果。**我们在表1中展示了主要研究成果。在不同系列的基础模型上，我们的采样算法在单次准确率和各类推理评估任务得分方面实现了显著且近乎普适的提升，例如在Phi-3.5-mini模型上HumanEval任务最高提升达+51.9%，在Qwen2.5-Math模型上MATH500任务提升达+25.2%。特别值得注意的是，在RL后训练领域的MATH500任务中，幂次采样的准确率与GRPO方法持平。此外，在领域外推理任务中，我们的算法在GPQA上与GRPO表现相当，而在HumanEval上甚至实现了最高+59.8%的超越。同样地，幂次采样在不可



|                              | MATH500      | HumanEval    | GPQA         | AlpacaEval2.0 |
|------------------------------|--------------|--------------|--------------|---------------|
| <b>Qwen2.5-Math-7B</b>       |              |              |              |               |
| Base                         | 0.496        | 0.329        | 0.278        | 1.61          |
| Low-temperature              | 0.690        | 0.512        | 0.353        | 2.09          |
| <b>Power Sampling (ours)</b> | <b>0.748</b> | <b>0.573</b> | <b>0.389</b> | <b>2.88</b>   |
| GRPO (MATH)                  | <b>0.785</b> | <b>0.537</b> | <b>0.399</b> | <b>2.38</b>   |
| <b>Qwen2.5-7B</b>            |              |              |              |               |
| Base                         | 0.498        | 0.329        | 0.278        | 7.05          |
| Low-temperature              | 0.628        | 0.524        | 0.303        | 5.29          |
| <b>Power Sampling (ours)</b> | <b>0.706</b> | <b>0.622</b> | <b>0.318</b> | <b>8.59</b>   |
| GRPO (MATH)                  | <b>0.740</b> | <b>0.561</b> | <b>0.354</b> | <b>7.62</b>   |
| <b>Phi-3.5-mini-instruct</b> |              |              |              |               |
| Base                         | 0.400        | 0.213        | 0.273        | 14.82         |
| Low-temperature              | 0.478        | 0.585        | 0.293        | <b>18.15</b>  |
| <b>Power Sampling (ours)</b> | <b>0.508</b> | <b>0.732</b> | <b>0.364</b> | <b>17.65</b>  |
| GRPO (MATH)                  | <b>0.406</b> | <b>0.134</b> | <b>0.359</b> | <b>16.74</b>  |

表 1: 幂次采样（本文方法）在不同模型家族与任务中匹配甚至超越GRPO。我们在MATH500、HumanEval、GPQA和AlpacaEval 2.0基准测试上评估了采样算法的性能。对本文方法与GRPO的得分均进行加粗标注，并在本文方法优于GRPO时添加下划线。跨模型实验表明，幂次采样在领域内推理任务（MATH500）上与GRPO性能相当，而在领域外任务中能够超越GRPO。

验证的 AlpacaEval 2.0 任务中持续优于基线，这表明我们的性能提升具有泛化性，能够超越可验证性领域的限制。

这个本质上简单却无需训练的采样算法所取得的惊人成功，揭示了现有基础模型潜在的推理能力。

### 5.3 分析

我们分析了幂采样的推理特性与GRPO之间的关系。表2展示了一个示例，更多案例见附录A.3。

**推理轨迹似然与置信度。**根据设计，幂采样旨在从基础模型中采样更高似然的序列。图4左侧绘制了基础模型、幂采样和GRPO在MATH500数据集上输出序列对数似然（按长度平均）的直方图，其中似然值相对于Qwen2.5-Math-7B基础模型计算。我们的方法按预期从基础模型更高似然区域采样，但仍保持明显分布广度；而GRPO样本则高度集中于最大似然峰值。

我们还绘制了MATH500响应在基础模型中的置信度，定义为下一词分布的平均负熵（不确定性）[Prabhudesai et al., 2025]:

$$\text{Conf}(x_{0:T}) = \frac{1}{T+1} \sum_{t=0}^T \sum_{x \in \mathcal{X}} p(x|x_{<t}) \log p(x|x_{<t}). \quad (13)$$

图4右侧显示，我们的方法与GRPO响应均从基础模型中相似的高置信区域采样，这些区域同样对应更高似然和正确推理。

**推理轨迹长度。**强化学习后训练的另一个特征是长形式推理[Guo et al., 2025]，其样本往往呈现更长响应。在MATH500上，Qwen2.5-Math-7B平均响应长度为600词元，而GRPO平均为671词元。值得注意的是，幂采样在未显式鼓励生成长文本情况下，仍达到679词元的平均长度，这一特性自然产生于采样过程。

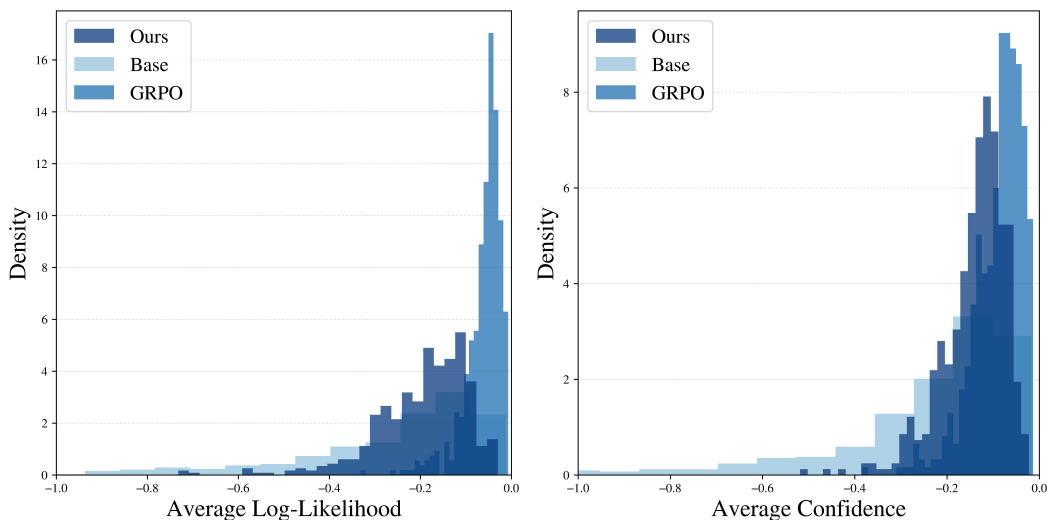


图 4: 基础模型 (Qwen2.5-Math-7B) 对 MATH500 响应的似然值与置信度。左图: 我们绘制了原始响应、幂采样响应与 GRPO 响应在 MATH500 数据集上的对数似然值 (相对于基础模型)。右图: 我们以相同方式展示了相对于基础模型的置信度。观测结果表明, GRPO 采样来自最高似然值与置信度区域, 幂采样紧随其后, 这与更高的实证准确率具有相关性。

**多样性与 pass@k 性能。** 观察图 4 中 GRPO 的尖峰集中分布与幂采样的广泛分布对比, 表明 GRPO 存在多样性坍缩, 而我们的采样器则保持多样性。这与“强化学习后训练以牺牲多样性为代价锐化基础模型分布”的观察一致 [Song et al., 2025]。为量化相对多样性, 我们绘制 pass@k 准确率曲线 (当 k 个样本中至少一个正确时判定问题解决)。图 5 显示: 与 GRPO 在大 k 值时性能下降不同, 幂采样在  $k > 1$  时显著优于 GRPO。此外, 我们的性能曲线在最终收敛前始终超越基础模型。特别地, 我们能在不损害多样性能前提下达到 GRPO 单样本水平 (其他领域结果见附录 A.2), 解决了强化学习后训练的长期缺陷。

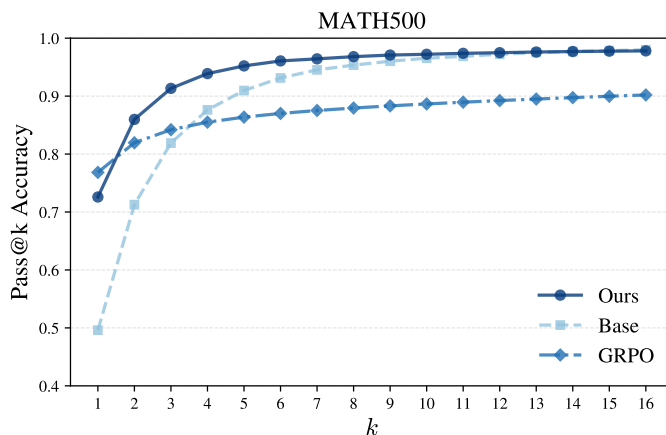


图 5: MATH500 数据集上的 Pass@k 性能对比。我们绘制了幂次采样 (本文方法) 与强化学习 (GRPO) 相对于基线模型 (Qwen2.5-Math-7B) 的 pass@k 准确率 (当 k 个样本中至少有一个正确时即判定为正确)。我们的性能曲线严格优于 GRPO 和基线模型, 且在高 k 值下的通过率与基线模型持平, 证明了生成结果的持续多样性。

**幂分布的影响。** 幂采样最重要的两个超参数是  $\alpha$  的选择和序列生成期间的 MCMC (重采样) 步数  $N_{\text{MCMC}}$ 。极端情况下,  $\alpha = 1.0$  直接从基础模型采样, 而  $\alpha \rightarrow \infty$  会确定性接受任何严格增加似然的重采样序列。虽然更高基础模型似然与更好推理相关 (图 4), 但直接优化似然对推理未必最优, 暗示存在理想中间  $\alpha$  值。

Filter an input list of strings only for ones that start with a given prefix. (Phi-3.5-mini-instruct: HumanEval)

| Method | Response                                                                                     | Passed |
|--------|----------------------------------------------------------------------------------------------|--------|
| Ours   | <pre>return [s for s in strings         if s.startswith(prefix)]</pre>                       | true   |
| GRPO   | <pre>return [string for string in strings if         string.startswith(f'{prefix}'*2)]</pre> | false  |

表 2: **HumanEval** 测试集上的样本响应: **Phi-3.5-mini-instruct** 模型。我们展示了一个示例, 其中我们的方法成功解决了一个简单编程问题, 而 GRPO 方法未能解决。

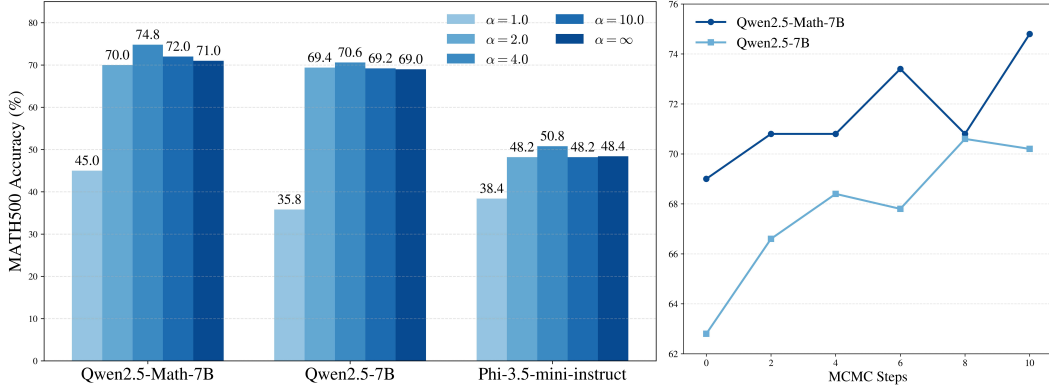


图 6: 超参数对幂采样的影响。左图: 我们绘制了不同 $\alpha$ 值下各模型族在MATH500数据集上的准确率。右图: 展示了随着MCMC步数增加, Qwen模型采用幂采样带来的准确率提升。

图6显示不同 $\alpha$ 值下MATH500准确率, 符合预期地, 中间值 $\alpha = 4.0$ 表现最佳。值得注意的是, 当 $\alpha \geq 2.0$ 后准确率保持相对稳定, 表明幂采样实践中对 $\alpha$ 选择具有鲁棒性。

测试时MCMC步数扩展。另一方面,  $N_{\text{MCMC}}$ 控制算法在推理时的计算量, 为测试时扩展提供自然维度。第4.3节提出的混合时间概念, 即充分采样目标分布所需MCMC步数。在我们的案例中, MCMC步数越少, 采样结果与目标分布 $p^\alpha$ 偏差越大。

图6展示了性能对 $N_{\text{MCMC}}$ 的依赖关系: 准确率在 $N_{\text{MCMC}} = 10$ 前持续增长, 之后趋于稳定(未绘制)。使用较少MCMC步数时,  $N_{\text{MCMC}} = 2$ 与 $N_{\text{MCMC}} = 10$ 间准确率差异约3-4%, 但采用两步采样相比零步的准确率跃升显著(3-4%)。

我们还可计算该方法相对GRPO的总词元生成量。根据式(12), 为生成长度 $T$ 的序列, 我们的采样器生成词元量为标准推理的 $\frac{1}{4B} \cdot N_{\text{MCMC}}T$ 倍。代入实验参数 $N_{\text{MCMC}} = 10$ 、 $T = 679$  (MATH500平均输出长度) 和 $B = 192$ , 幂采样推理的词元量是标准推理的**8.84**倍。由于GRPO训练时每个样本需多轮生成, 本方法推理成本约等同于GRPO一个训练周期(假设每样本8轮生成且数据集规模相同)。但典型情况下, GRPO周期成本仍更高, 因其使用16轮生成且训练集大于MATH500。

## 6 结论

在本工作中, 我们提出了一种无需额外训练或依赖外部信号的算法, 可直接从基础模型中进行采样, 其单次推理性能与最先进的强化学习后训练算法相当, 有时甚至更优。通过讨论强化学习的分布锐化现象, 我们论证了将幂分布定义为推理目标分布的理论价值。虽然精确的幂分布采样具有计算复杂性, 但我们结合经典MCMC技术与自回归生成的序列结构, 提出了幂采样算法, 并验证了其卓越的实证性能。

研究表明，基础模型在采样阶段的潜力尚未被充分挖掘，同时揭示了基础模型高似然区域与强大推理能力之间的紧密关联。通过在采样阶段投入更多计算资源，并深化对基础模型能力的理解，这一研究为突破可验证性边界、拓展推理能力范围提供了富有前景的技术路径。

## 7 致谢

A.K. 衷心感谢保罗与黛西·索罗斯基金会、NDSEG奖学金以及肯普纳研究所的支持。

## 参考文献

- Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. Deepseek-r1: Incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025. 1, 3, 9
- Jingcheng Hu, Yinmin Zhang, Qi Han, Daxin Jiang, Xiangyu Zhang, and Heung-Yeung Shum. Open-reasoner-zero: An open source approach to scaling up reinforcement learning on the base model. *arXiv preprint arXiv:2503.24290*, 2025. 1, 3
- Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. *arXiv preprint arXiv:2103.03874*, 2021. 1
- Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, Thomas Hubert, Peter Choy, Cyprien de Masson d’Autume, Igor Babuschkin, Xinyun Chen, Po-Sen Huang, Johannes Welbl, Sven Gowal, Alexey Cherepanov, James Molloy, Daniel Mankowitz, Esme Sutherland Robson, Pushmeet Kohli, Nando de Freitas, Koray Kavukcuoglu, and Oriol Vinyals. Competition-level code generation with AlphaCode. *arXiv preprint arXiv:2203.07814*, 2022. 1
- David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R Bowman. GPQA: A graduate-level google-proof q&a benchmark. In *First Conference on Language Modeling*, 2024. 1, 8
- Andre He, Daniel Fried, and Sean Welleck. Rewarding the unlikely: Lifting GRPO beyond distribution sharpening. *arXiv preprint arXiv:2506.02355*, 2025. 1
- Rulin Shao, Shuyue Stella Li, Rui Xin, Scott Geng, Yiping Wang, Sewoong Oh, Simon Shao-lei Du, Nathan Lambert, Sewon Min, Ranjay Krishna, Yulia Tsvetkov, Hannaneh Hajishirzi, Pang Wei Koh, and Luke Zettlemoyer. Spurious rewards: Rethinking training signals in RLVR. *arXiv preprint arXiv:2506.10947*, 2025. 1, 3, 8
- Yang Yue, Zhiqi Chen, Rui Lu, Andrew Zhao, Zhaokai Wang, Shiji Song, and Gao Huang. Does reinforcement learning really incentivize reasoning capacity in llms beyond the base model? *arXiv preprint arXiv:2504.13837*, 2025. URL <https://arxiv.org/abs/2504.13837>. 1
- Yuda Song, Julia Kempe, and Rémi Munos. Outcome-based exploration for LLM reasoning. *arXiv preprint arXiv:2509.06941*, 2025. URL <https://arxiv.org/abs/2509.06941>. 1, 10
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. Deepseek-math: Advancing mathematical reasoning through step-by-step exploration. *arXiv preprint arXiv:2404.01140*, 2024. 2, 3

- Mihir Prabhudesai, Lili Chen, Alex Ippoliti, Katerina Fragkiadaki, Hao Liu, and Deepak Pathak. Maximizing confidence alone improves reasoning. *arXiv preprint arXiv:2505.22660*, 2025. URL <https://arxiv.org/abs/2505.22660>. 2, 3, 9
- Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. In *NeurIPS*, volume 35, pages 27730–27744, 2022. 3
- Nathan Lambert, Jacob Morrison, Valentina Pyatkin, Shengyi Huang, Hamish Ivison, Faeze Brahman, Lester James V Miranda, Alisa Liu, Nouha Dziri, Shane Lyu, et al. Tulu 3: Pushing frontiers in open language model post-training. *arXiv preprint arXiv:2411.15124*, 2024. 3
- Weihao Zeng, Yuzhen Huang, Qian Liu, Wei Liu, Keqing He, Zejun Ma, and Junxian He. Simplerlzo: Investigating and taming zero reinforcement learning for open base models in the wild. *arXiv preprint arXiv:2503.18892*, 2025. URL <https://arxiv.org/abs/2503.18892>. 3
- Xuandong Zhao, Zhewei Kang, Aosong Feng, Sergey Levine, and Dawn Song. Learning to reason without external rewards. *arXiv preprint arXiv:2505.19590*, 2025. URL <https://arxiv.org/abs/2505.19590>. 3
- Stephen Zhao, Rob Brekelmans, Alireza Makhzani, and Roger Grosse. Probabilistic inference in language models via twisted sequential monte carlo. *arXiv preprint arXiv:2404.17546*, 2024. URL <https://arxiv.org/abs/2404.17546>. 3
- Nicolas Chopin. Central limit theorem for sequential monte carlo methods and its application to bayesian inference. *The Annals of Statistics*, 32(6): 2385–2411, 2004. doi: 10.1214/009053604000000615. URL <https://projecteuclid.org/journals/annals-of-statistics/volume-32/issue-6/Central-limit-theorem-for-sequential-Monte-Carlo-methods-and-its/10.1214/009053604000000698.full>. 3
- Gonalo R. A. Faria, Sweta Agrawal, Ant3nio Farinhas, Ricardo Rei, Jos3 G. C. de Souza, and Andr3 F. T. Martins. Quest: Quality-aware metropolis-hastings sampling for machine translation. In *NeurIPS*, 2024. URL [https://proceedings.neurips.cc/paper\\_files/paper/2024/file/a221d22ff6a33599142c8299c7ed06bb-Paper-Conference.pdf](https://proceedings.neurips.cc/paper_files/paper/2024/file/a221d22ff6a33599142c8299c7ed06bb-Paper-Conference.pdf). 3
- Radford M. Neal. Annealed importance sampling. *arXiv preprint physics/9803008*, 1998. URL <https://arxiv.org/abs/physics/9803008>. 3
- Krzysztof Łatuszyński, Matthew T. Moores, and Timoth3e Stumpf-F3tizon. Mcmc for multi-modal distributions. *arXiv preprint arXiv:2501.05908*, 2025. URL <https://arxiv.org/abs/2501.05908v1>. 3
- Yilun Du, Conor Durkan, Robin Strudel, Joshua B Tenenbaum, Sander Dieleman, Rob Fergus, Jascha Sohl-Dickstein, Arnaud Doucet, and Will Sussman Grathwohl. Reduce, reuse, recycle: Compositional generation with energy-based diffusion models and mcmc. In *International conference on machine learning*, pages 8489–8510. PMLR, 2023. 3
- Sunwoo Kim, Minkyu Kim, and Dongmin Park. Test-time alignment of diffusion models without reward over-optimization. *arXiv preprint arXiv:2501.05803*, 2025. URL <https://arxiv.org/abs/2501.05803>. 3



- Aayush Karan, Kulin Shah, and Sitan Chen. Reguidance: A simple diffusion wrapper for boosting sample quality on hard inverse problems. *arXiv preprint arXiv:2506.10955*, 2025. URL <https://arxiv.org/abs/2506.10955>. 3
- Yanwei Wang, Lirui Wang, Yilun Du, Balakumar Sundaralingam, Xuning Yang, Yu-Wei Chao, Claudia Pérez-D’ Arpino, Dieter Fox, and Julie Shah. Inference-time policy steering through human interactions. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*, pages 15626–15633. IEEE, 2025. 3
- Lingkai Kong, Yuanqi Du, Wenhao Mu, Kirill Neklyudov, Valentin De Bortoli, Dongxia Wu, Haorui Wang, Aaron M. Ferber, Yian Ma, Carla P. Gomes, and Chao Zhang. Diffusion models as constrained samplers for optimization with unknown constraints. In Yingzhen Li, Stephan Mandt, Shipra Agrawal, and Emtiyaz Khan, editors, *Proceedings of The 28th International Conference on Artificial Intelligence and Statistics*, volume 258 of *Proceedings of Machine Learning Research*, pages 4582–4590. PMLR, 2025. URL <https://proceedings.mlr.press/v258/kong25b.html>. 3
- Xiangcheng Zhang, Haowei Lin, Haotian Ye, James Zou, Jianzhu Ma, Yitao Liang, and Yilun Du. Inference-time scaling of diffusion models through classical search. *arXiv preprint arXiv:2505.23614*, 2025. 3
- Malcolm Sambridge. A parallel tempering algorithm for probabilistic sampling and optimization. *Geophysical Journal International*, 196(1):357–374, 2014. doi: 10.1093/gji/ggt374. URL <https://academic.oup.com/gji/article/196/1/357/585739>. 3
- Marta Skreta, Tara Akhound-Sadegh, Viktor Ohanesian, Roberto Bondesan, Alán Aspuru-Guzik, Arnaud Doucet, Rob Brekelmans, Alexander Tong, and Kirill Neklyudov. Feynman-kac correctors in diffusion: Annealing, guidance, and product of experts. *arXiv preprint arXiv:2503.02819*, 2025. URL <https://arxiv.org/abs/2503.02819>. 3
- Yanbo Xu, Yu Wu, Sungjae Park, Zhizhuo Zhou, and Shubham Tulsiani. Temporal score rescaling for temperature sampling in diffusion and flow models. *arXiv preprint arXiv:2510.01184*, 2025. URL <https://arxiv.org/abs/2510.01184>. 3
- Tomas Geffner, Kieran Didi, Zuobai Zhang, Danny Reidenbach, Zhonglin Cao, Jason Yim, Mario Geiger, Christian Dallago, Emine Kucukbenli, and Arash Vahdat. Proteina: Scaling flow-based protein structure generative models. *arXiv preprint arXiv:2503.00710*, 2025. URL <https://arxiv.org/abs/2503.00710>. 3
- Pei-Hsin Wang, Sheng-Iou Hsieh, Shih-Chieh Chang, Yu-Ting Chen, Jia-Yu Pan, Wei Wei, and Da-Chang Juan. Contextual temperature for language modeling. *arXiv preprint arXiv:2012.13575*, 2020. URL <https://arxiv.org/abs/2012.13575>. 4
- Marvin Li, Aayush Karan, and Sitan Chen. Blink of an eye: A simple theory for feature localization in generative models. *arXiv preprint arXiv:2502.00921*, 2025. URL <https://arxiv.org/abs/2502.00921>. 5, 16
- Marah Abidin, Jyoti Aneja, Harkirat Behl, Sébastien Bubeck, Ronen Eldan, Suriya Gunasekar, Michael Harrison, Russell J. Hewett, Mojan Javaheripi, Piero Kauffmann, James R. Lee, Yin Tat Lee, Yuanzhi Li, Weishung Liu, Caio C. T. Mendes, Anh Nguyen, Eric Price, Gustavo de

- Rosa, Olli Saarikivi, Adil Salim, Shital Shah, Xin Wang, Rachel Ward, Yue Wu, Dingli Yu, Cyril Zhang, and Yi Zhang. Phi-4 technical report. *arXiv preprint arXiv:2412.08905*, 2024. URL <https://arxiv.org/abs/2412.08905>. 5, 8, 16
- Nicholas Metropolis, Arianna W. Rosenbluth, Marshall N. Rosenbluth, Augusta H. Teller, and Edward Teller. Equation of state calculations by fast computing machines. *Journal of Chemical Physics*, 21(6):1087–1092, 1953. doi: 10.1063/1.1699114. 5
- Radford M Neal. Probabilistic inference using markov chain monte carlo methods. *Department of Computer Science, University of Toronto (review paper / technical report)*, 1993. 6
- Reza Gheissari, Eyal Lubetzky, and Yuval Peres. Exponentially slow mixing in the mean-field swendsen–wang dynamics. *arXiv preprint arXiv:1702.05797*, 2017. 6
- Afonso S. Bandeira, Antoine Maillard, Richard Nickl, and Sven Wang. On free energy barriers in gaussian priors and failure of cold start mcmc for high-dimensional unimodal distributions. *arXiv preprint arXiv:2209.02001*, 2022. URL <https://arxiv.org/abs/2209.02001>. 6
- Scott C. Schmidler and Dawn B. Woodard. Lower bounds on the convergence rates of adaptive mcmc methods. Technical report, Duke University / Cornell University, 2013. URL [https://www2.stat.duke.edu/~scs/Papers/AdaptiveLowerBounds\\_AS.pdf](https://www2.stat.duke.edu/~scs/Papers/AdaptiveLowerBounds_AS.pdf). 6
- Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let’s verify step by step. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=v8LOpN6E0i>. 8
- M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, A. Ray, R. Puri, G. Krueger, M. Petrov, H. Khlaaf, G. Sastry, P. Mishkin, B. Chan, S. Gray, N. Ryder, M. Pavlov, A. Power, L. Kaiser, M. Bavarian, C. Winter, P. Tillet, F. P. Such, D. Cummings, M. Plappert, F. Chantzis, E. Barnes, A. Herbert-Voss, W. H. Guss, A. Nichol, A. Paino, N. Tezak, J. Tang, I. Babuschkin, S. Balaji, S. Jain, W. Saunders, C. Hesse, A. N. Carr, J. Leike, J. Achiam, V. Misra, E. Morikawa, A. Radford, M. Knight, M. Brundage, M. Murati, K. Mayer, P. Welinder, B. McGrew, D. Amodei, S. McCandlish, I. Sutskever, and W. Zaremba. Evaluating large language models trained on code. *CoRR*, abs/2107.03374, 2021. URL <https://arxiv.org/abs/2107.03374>. 8
- Yann Dubois, Balázs Galambosi, Percy Liang, and Tatsunori B. Hashimoto. Length-controlled alpacaeval: A simple way to debias automatic evaluators. *arXiv preprint arXiv:2404.04475*, 2024. URL <https://arxiv.org/abs/2404.04475>. 8

## A 附录

### A.1 补充理论讨论

本节我们将对功率采样能削弱那些将输出困于低似然未来的标记、而低温采样则无法做到这一现象进行更严格的形式化描述。

**Proposition 2** (非正式描述). 幂次采样会提升那些支持集较小但后续词元出现概率较高的词元权重, 而低温采样则会提升那些支持集较大但后续词元出现概率较低的词元权重。

**Definition 2.** 在本节后续内容中, 固定一个前缀  $x_{0:t-1}$ 。若在条件下一词元分布下满足  $\sum_{x>t} p(x_0, \dots, x_t, \dots, x_T) = \varepsilon$ , 则称  $x_t$  具有 **边际权重**  $\varepsilon$ 。

我们考虑一个简化的“关键窗口”或“关键标记”现象模型 [Li et al., 2025, Abdin et al., 2024], 该现象指那些能显著影响最终生成质量的中间标记。我们区分了导向高似然未来与低似然未来的两类关键标记。

**Definition 3.** 在一种极端情况下, 若一个关键性令牌将其全部边际权重  $\varepsilon$  集中于单一未来路径 (单点支撑), 即仅存在一个  $x > t$  的选择使得  $p(x_0, \dots, x_t, \dots, x_T)$  不为零, 则该令牌能最大程度地诱导出高似然性续写。我们称此类令牌为**正向关键令牌**。

**Definition 4.** 在另一个极端情况下, 若某个关键令牌的全部边际权重  $\varepsilon$  均匀分布在  $N$  个后续补全上, 则该令牌会最小化任何未来序列的可能性。换言之, 存在  $N$  个补全序列  $x > t$ , 使得所有  $p(x_0, \dots, x_t, \dots, x_T)$  均以  $\frac{\varepsilon}{N}$  的概率非零。我们将此类令牌称为**负向关键令牌**。

我们关于高/低似然未来的简化模型研究了在给定采样分布下, 正向关键标记何时能优先于负向关键标记被选中。特别地, 我们证明即使负向关键标记具有更高的边际权重, 功率采样仍能提升正向关键标记的选中概率, 而低温采样在此情况下总是会优先选择负向关键标记。

当然, 当正向关键标记本身具有更高边际权重时, 功率采样与低温采样都会优先选择它。

**Proposition 3.** 设  $x_t$  为具有边际权重  $\varepsilon$  的正向关键标记,  $x'_t$  为具有边际权重  $\varepsilon'$  和支持集  $N$  的负向关键标记。若满足

$$\frac{\varepsilon'}{N^{1-1/\alpha}} < \varepsilon < \varepsilon', \quad (14)$$

则  $x_t$  的未来似然将高于  $x'_t$  的任何未来似然值。此外, 幂次采样会提升  $x_t$  相对于  $x'_t$  的权重, 而低温采样则会提升  $x'_t$  相对于  $x_t$  的权重。

证明. 由于  $\alpha \geq 1$ , 可得

$$\frac{\varepsilon'}{N^{1-1/\alpha}} > \frac{\varepsilon'}{N} \quad (15)$$

因此  $\varepsilon > \frac{\varepsilon'}{N}$ , 这表明  $x_t$  的未来补全似然高于  $x'_t$  (即正负关键标记的划分是自洽的)。

当  $\varepsilon < \varepsilon'$  时, 在低温分布下  $x_t$  与  $x'_t$  的相对边际权重为  $\varepsilon^\alpha$  和  $\varepsilon'^\alpha$ , 因此选择  $x_t$  的概率相对于  $x'_t$  会被削弱。然而对于功率分布, 相对边际权重为  $p_{\text{pow}}(x_t|x_{<t}) = \varepsilon^\alpha$  和  $p_{\text{pow}}(x'_t|x_{<t}) = \frac{\varepsilon'^\alpha}{N^{\alpha-1}}$ 。此时只要满足  $\varepsilon^\alpha > \frac{\varepsilon'^\alpha}{N^{\alpha-1}} \iff \varepsilon > \frac{\varepsilon'}{N^{1-1/\alpha}}$ , 标记  $x_t$  相对于  $x'_t$  就会被增强。

换言之, 在分布  $p$  下  $x_t$  的边际权重可以小于  $x'_t$  的权重, 但只要  $x_t$  的补全比  $x'_t$  的任何个体补全具有更高似然, 功率采样就会优先选择  $x_t$  而非  $x'_t$ 。□

### A.2 多领域下的Pass@k准确率

本节我们绘制了power sampling、GRPO和基础模型 (Qwen2.5-Math-7B) 在MATH500、GPQA和HumanEval数据集上的pass@k性能曲线, 以证明我们的采样算法在单次推理和多

次推理场景下均能保持优异性能，同时维持回答多样性。对于MATH500和GPQA数据集，power sampling的参数设为 $\alpha = 4.0$ ；对于HumanEval数据集则设为 $\alpha = 1.67$ （该温度参数在较小 $k$ 值时表现略优）。在所有情况下（包括GRPO的域内和域外场景），当 $k > 1$ 时，power sampling的pass@ $k$ 性能几乎全面优于GRPO和基础模型；当 $k$ 值较大时，其表现至少与基础模型上限持平，甚至有所超越。

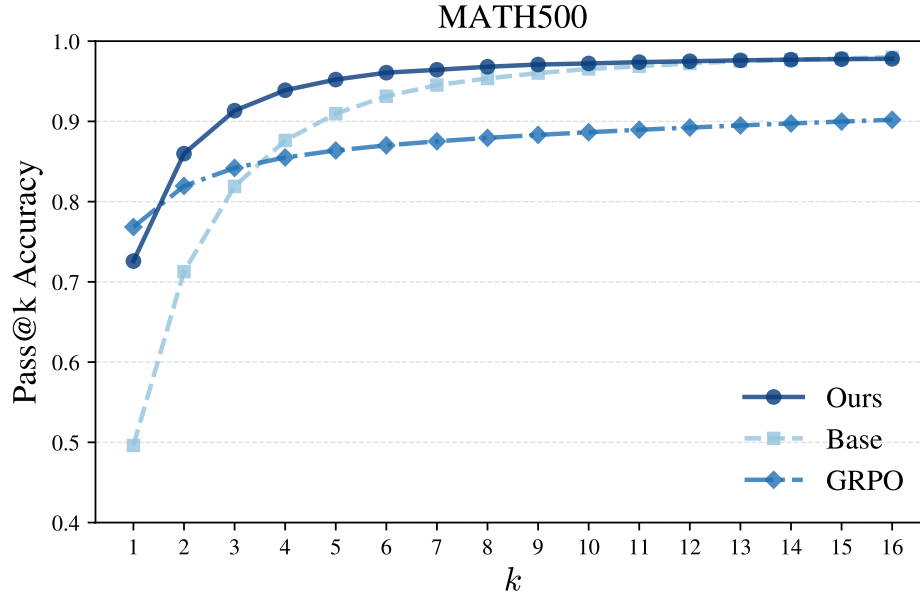


图 7: MATH500 数据集上 Pass@ $k$  性能表现 (Qwen2.5-Math-7B 模型)。

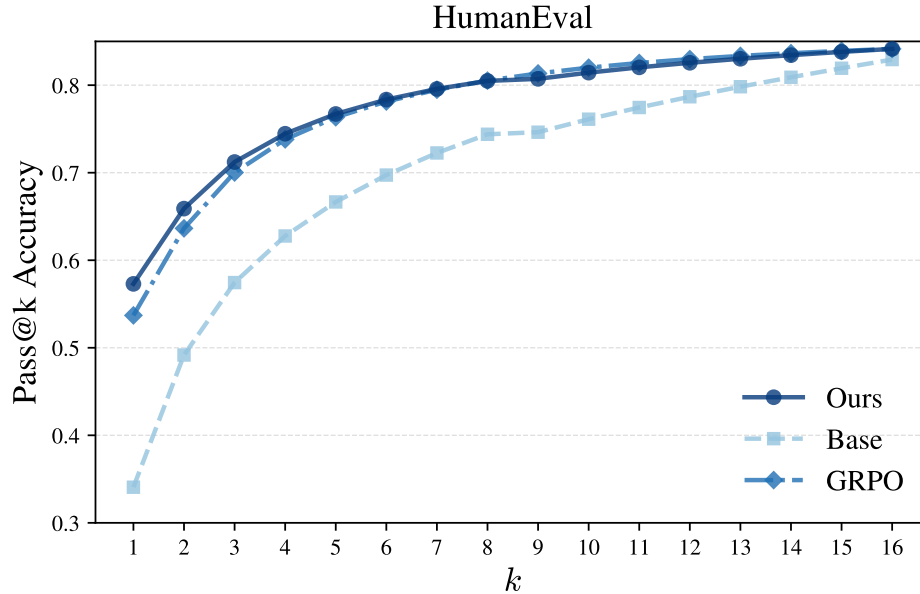


图 8: Qwen2.5-Math-7B 在 HumanEval 上的 Pass@ $k$  性能表现。

需要特别指出的是，这些曲线显示的多样性损失在不同基准测试中存在显著差异。MATH500和GPQA数据明确显示，即使对于较小的 $k$ 值，GRPO的pass@ $k$ 性能和多样性

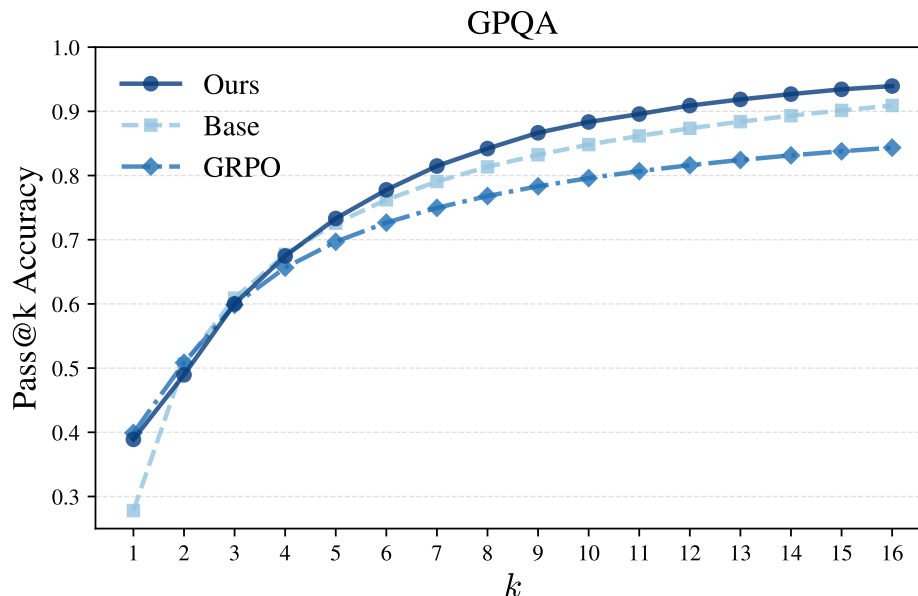


图 9: GPQA 数据集上 Qwen2.5-Math-7B 模型的 Pass@k 性能表现.

也明显较低；而在HumanEval测试中，GRPO的pass@k表现优于基础模型直至 $k = 16$ 。我们推测这可能是因为MATH500和GPQA基于“单一”答案评分，而HumanEval则基于代码补全任务（允许多种解决方案），这表明多样性降低的现象可能出现，但需要更大的 $k$ 值才会显现。

### A.3 更多定性分析示例

| Can you solve the following math problem? Write $\frac{3}{20}$ as a decimal. Please reason step by step, and put your final answer within <code>\boxed{\{\}}</code> . Correct answer: <code>0.15</code> |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| System                                                                                                                                                                                                  | Answer                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| Ours                                                                                                                                                                                                    | <p>To convert the fraction <math>\frac{3}{20}</math> to a decimal, we can follow these steps:</p> <ol style="list-style-type: none"> <li>1. Identify the numerator and the denominator of the fraction. In this case, the numerator is 3 and the denominator is 20.</li> <li>2. Divide the numerator by the denominator. In this case, we divide 3 by 20.</li> <li>3. Perform the division. The result is 0.15.</li> </ol> <p>Therefore, the fraction <math>\frac{3}{20}</math> as a decimal is <code>0.15</code>.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| GRPO                                                                                                                                                                                                    | <p>To convert the fraction <math>\frac{3}{20}</math> to a decimal, we can follow these steps:</p> <ol style="list-style-type: none"> <li>1. <b>Understand the fraction:</b> The fraction <math>\frac{3}{20}</math> means 3 divided by 20.</li> <li>2. <b>Perform the division:</b> We need to divide 3 by 20. Since 3 is less than 20, we will add a decimal point and zeros to 3 to make the division possible.</li> <li>3. <b>Set up the division:</b> Write 3 as 3.000 (adding three zeros for precision).</li> <li>4. <b>Divide:</b> - 20 goes into 30 once (since <math>20 \times 1 = 20</math>). - Subtract 20 from 30 to get a remainder of 10. - Bring down the next 0 to make it 100. - 20 goes into 100 five times (since <math>20 \times 5 = 100</math>). - Subtract 100 from 100 to get a remainder of 0.</li> <li>5. <b>Combine the results:</b> The quotient from the division is 0.15.</li> </ol> <p>Therefore, the decimal representation of <math>\frac{3}{20}</math> is <code>0.15</code>.</p> |

表 3: MATH500 对比实验：Qwen2.5-Math-7B 模型。我们展示了一个本方法与GRPO均能正确求解的示例。



*The Fib4 number sequence is a sequence similar to the Fibonacci sequence defined as:  $fib4(0)=0$ ,  $fib4(1)=0$ ,  $fib4(2)=2$ ,  $fib4(3)=0$ , and for  $n \geq 4$ ,  $fib4(n) = fib4(n-1) + fib4(n-2) + fib4(n-3) + fib4(n-4)$ . Write a function to efficiently compute the  $n$ -th element without recursion.*

| System | Completion                                                                                                                                                                                                                                                                                                                                                                                                                                                                 | Passed |
|--------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------|
| Ours   | <pre> # Base cases if n == 0:     return 0 elif n == 1:     return 0 elif n == 2:     return 2 elif n == 3:     return 0  # Initialize the first four elements fib4_values = [0, 0, 2, 0]  # Compute the rest of the sequence iteratively for i in range(4, n+1):     next_value = (fib4_values[i-1] + fib4_values[i-2] +                   fib4_values[i-3] + fib4_values[i-4])     fib4_values.append(next_value)  # Return the n-th element return fib4_values[n]</pre> | true   |
| GRPO   | <pre> a, b, c, d = 0, 0, 2, 0 for _ in range(n):     a, b, c, d = b, c, d, a + b + c + d return d</pre>                                                                                                                                                                                                                                                                                                                                                                    | false  |

表 4: Phi-3.5-mini-instruct 模型在 HumanEval 基准上的性能对比

| Can you solve the following math problem? What integer $n$ satisfies $0 \leq n < 18$ and                                 |                                                                                                                                                                                                                                                               |
|--------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $n \equiv -11213141 \pmod{18} ?$                                                                                         |                                                                                                                                                                                                                                                               |
| Please reason step by step, and put your final answer within <code>\boxed{\{\}}</code> . Correct answer: <code>13</code> |                                                                                                                                                                                                                                                               |
| Method                                                                                                                   | Answer                                                                                                                                                                                                                                                        |
| Ours                                                                                                                     | First, we perform the division of 11213141 by 18:                                                                                                                                                                                                             |
|                                                                                                                          | $11213141 \div 18 \approx 622952.277777 \dots$                                                                                                                                                                                                                |
|                                                                                                                          | The integer part of the quotient is 622952. Now, we multiply 622952 by 18 to find the product:                                                                                                                                                                |
|                                                                                                                          | $622952 \times 18 = 11213136$                                                                                                                                                                                                                                 |
|                                                                                                                          | Next, we subtract this product from 11213141 to find the remainder:                                                                                                                                                                                           |
|                                                                                                                          | $11213141 - 11213136 = 5$                                                                                                                                                                                                                                     |
|                                                                                                                          | So, the remainder when 11213141 is divided by 18 is 5. This means:                                                                                                                                                                                            |
|                                                                                                                          | $11213141 \equiv 5 \pmod{18}$                                                                                                                                                                                                                                 |
|                                                                                                                          | Therefore:                                                                                                                                                                                                                                                    |
|                                                                                                                          | $-11213141 \equiv -5 \pmod{18}$                                                                                                                                                                                                                               |
| GRPO                                                                                                                     | Since we need a non-negative remainder, we convert $-5$ to a positive equivalent by adding 18:                                                                                                                                                                |
|                                                                                                                          | $-5 + 18 = 13$                                                                                                                                                                                                                                                |
|                                                                                                                          | Thus:                                                                                                                                                                                                                                                         |
|                                                                                                                          | $-11213141 \equiv 13 \pmod{18}$                                                                                                                                                                                                                               |
|                                                                                                                          | The integer $n$ that satisfies $0 \leq n < 18$ and $n \equiv -11213141 \pmod{18}$ is: <code>13</code> .                                                                                                                                                       |
|                                                                                                                          | Step 1: Find the remainder when 11213141 is divided by 18. We can use the property of modular arithmetic that states $a \equiv b \pmod{m}$ if and only if $a \bmod m = b \bmod m$ . So, we need to compute $11213141 \bmod 18$ .                              |
|                                                                                                                          | To simplify this, we can use the fact that the remainder of a number when divided by 18 is the same as the remainder of the sum of its digits when divided by 18 (although this is not always the most efficient method, it can be used here for simplicity). |
|                                                                                                                          | First, let's sum the digits of 11213141:                                                                                                                                                                                                                      |
|                                                                                                                          | $1 + 1 + 2 + 1 + 3 + 1 + 4 + 1 = 14.$                                                                                                                                                                                                                         |
|                                                                                                                          | Now, we need to find the remainder when 14 is divided by 18. Since 14 is less than 18, the remainder is simply 14. Therefore,                                                                                                                                 |
|                                                                                                                          | $11213141 \equiv 14 \pmod{18}.$                                                                                                                                                                                                                               |
|                                                                                                                          | Step 2: Find the remainder when $-11213141$ is divided by 18. Since $11213141 \equiv 14 \pmod{18}$ , it follows that                                                                                                                                          |
|                                                                                                                          | $-11213141 \equiv -14 \pmod{18}.$                                                                                                                                                                                                                             |
|                                                                                                                          | We need to convert $-14$ to a positive remainder between 0 and 17. To do this, we add 18 to $-14$ :                                                                                                                                                           |
|                                                                                                                          | $-14 + 18 = 4.$                                                                                                                                                                                                                                               |
|                                                                                                                          | Therefore,                                                                                                                                                                                                                                                    |
|                                                                                                                          | $-11213141 \equiv 4 \pmod{18}.$                                                                                                                                                                                                                               |
|                                                                                                                          | The integer $n$ that satisfies $0 \leq n < 18$ and $n \equiv -11213141 \pmod{18}$ is <code>4</code> .                                                                                                                                                         |

表 5: MATH500 基准测试中我们的采样算法与 GRPO 在 Qwen2.5-Math-7B 模型上的对比示例。此处展示了一个 GRPO 得出错误答案而我们的采样算法成功解决的案例。我们的采样答案采用了完全不同的求解方法。